

DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

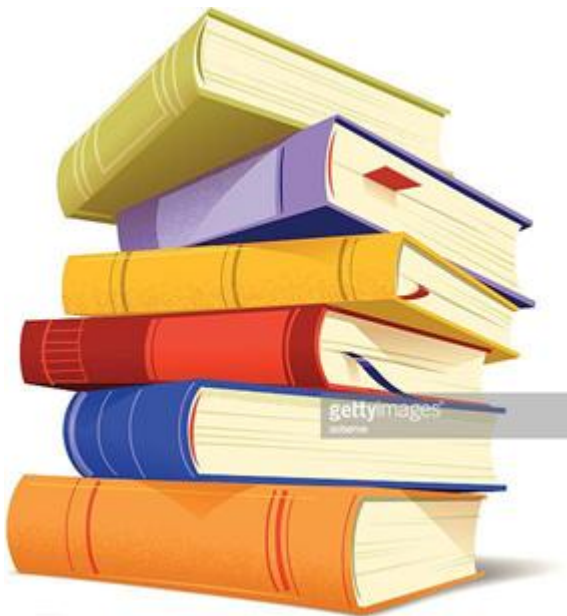
www.hoasen.edu.vn

Chương 2



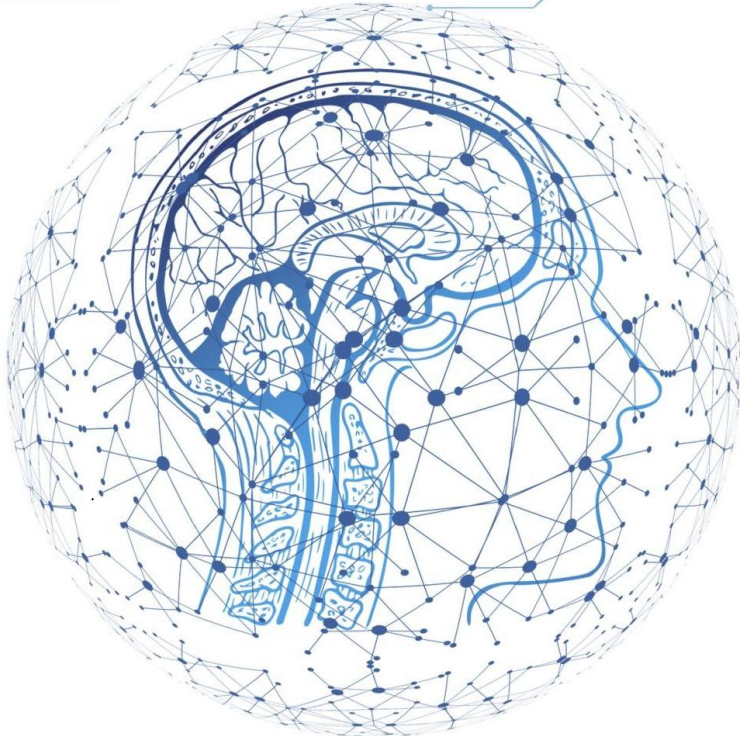
Các thành phần toán học
nền tảng của mạng nơ-ron

Học sâu là gì?



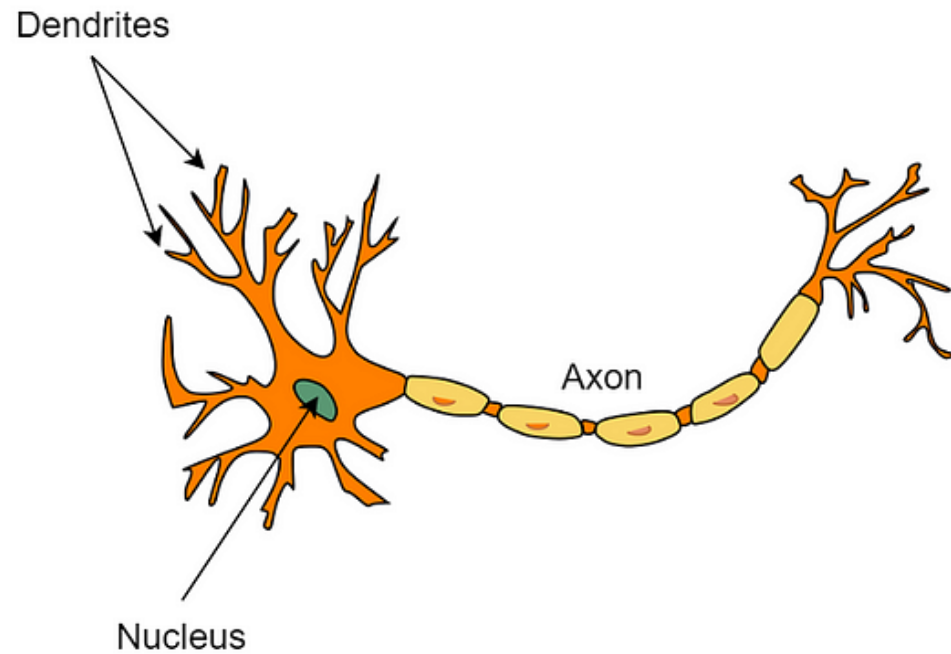
DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU

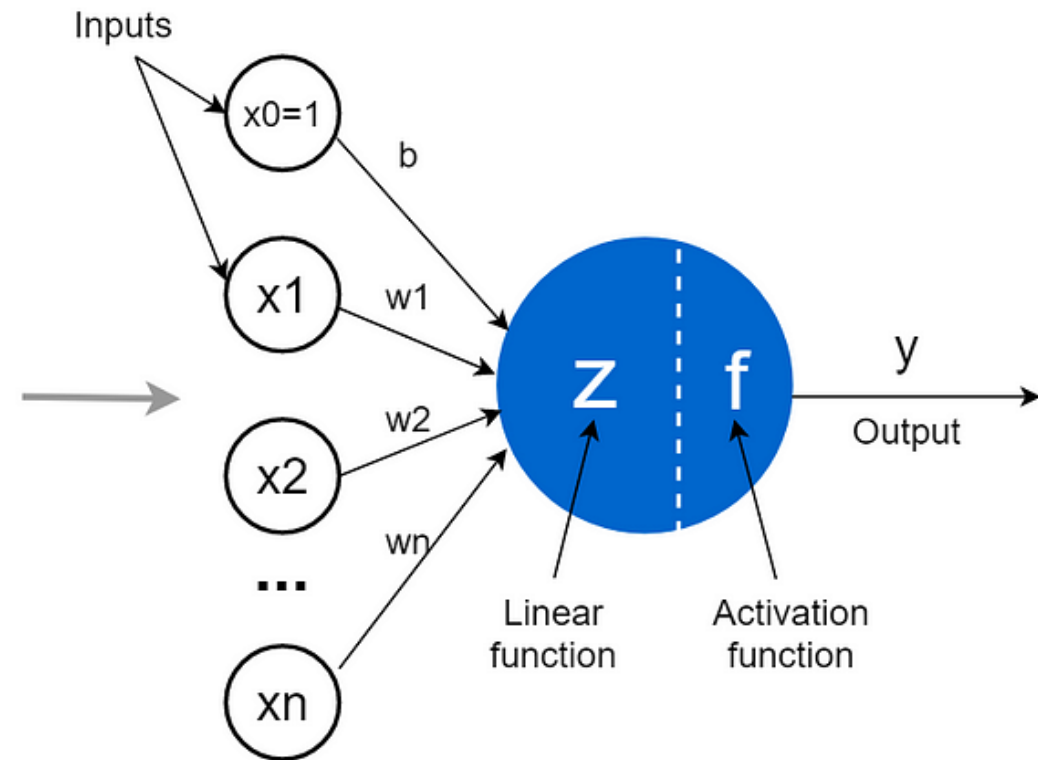


Perceptron

Neuron và Perceptron

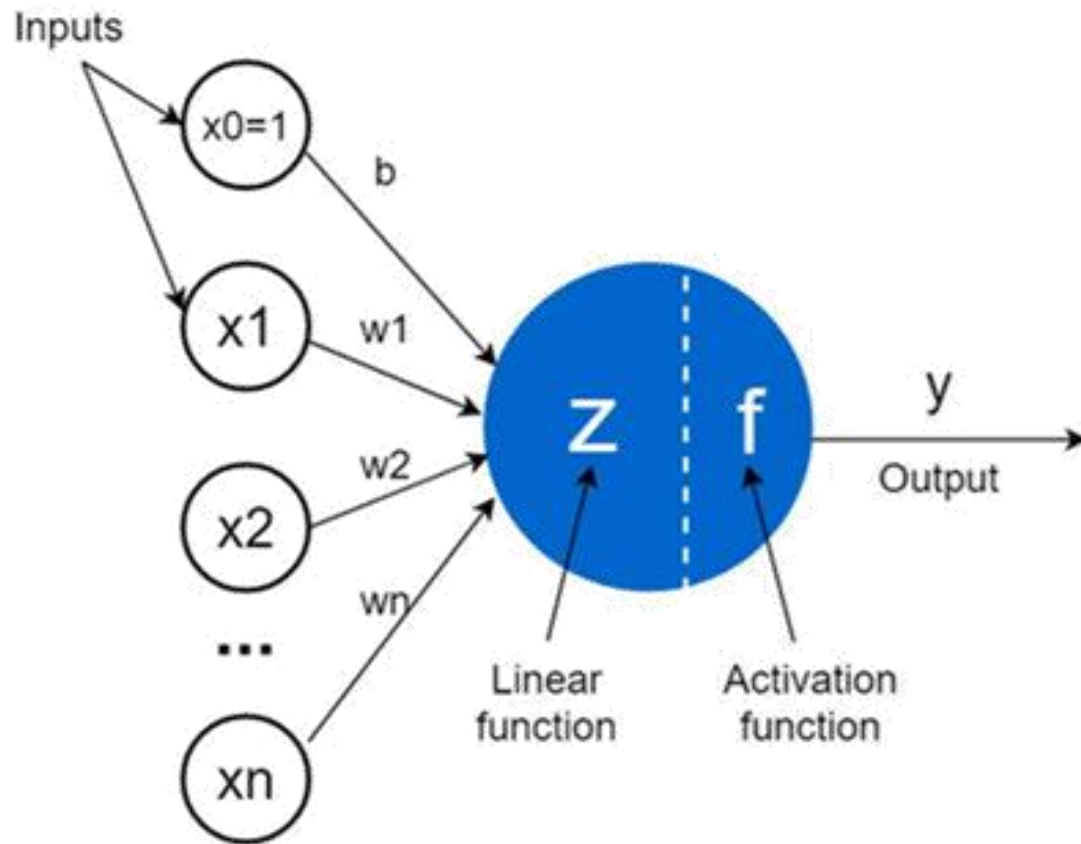


Neuron



Perceptron (Frank Rosenblatt, 1958)

Perceptron là gì?



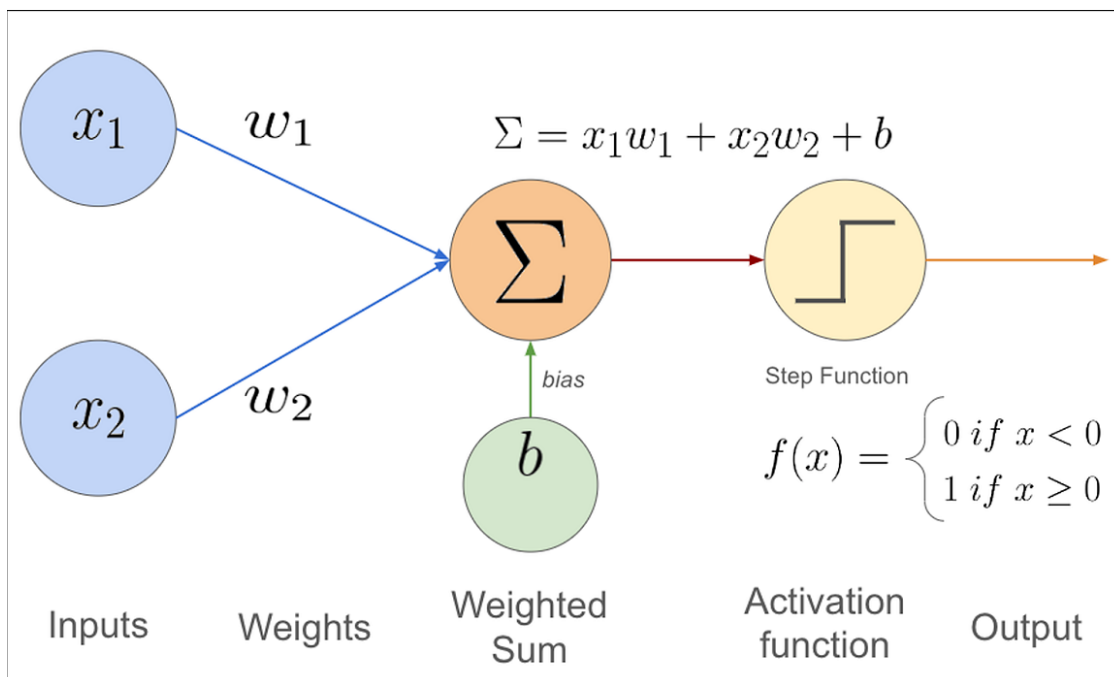
■ Perceptron:

- Mô hình đơn giản của neuron sinh học
- Khối xây dựng cơ bản của mạng nơ-ron
- Dùng để phân loại nhị phân

■ Ý tưởng:

- Nhận input $\rightarrow$ xử lý $\rightarrow$ đưa ra output

Cấu trúc Perceptron

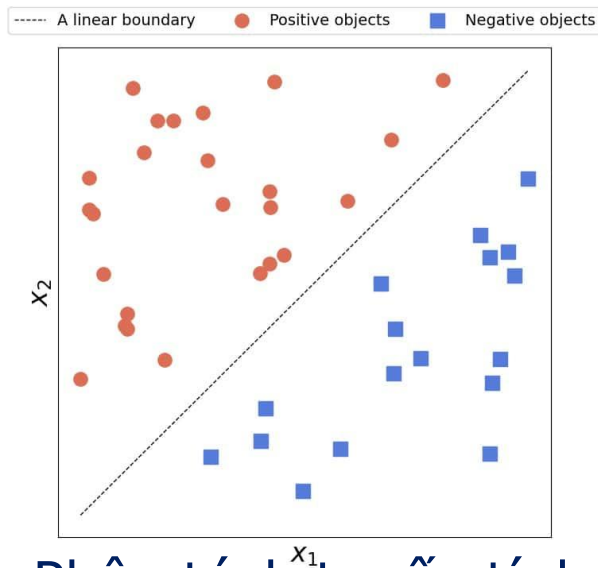


- Input: dữ liệu đầu vào
- Weight (trọng số): cho biết mức độ quan trọng của input
- Bias (độ lệch): điều chỉnh ngưỡng phân loại
- Activation function (hàm kích hoạt): quyết định output là gì

Vai trò & hạn chế của Perceptron

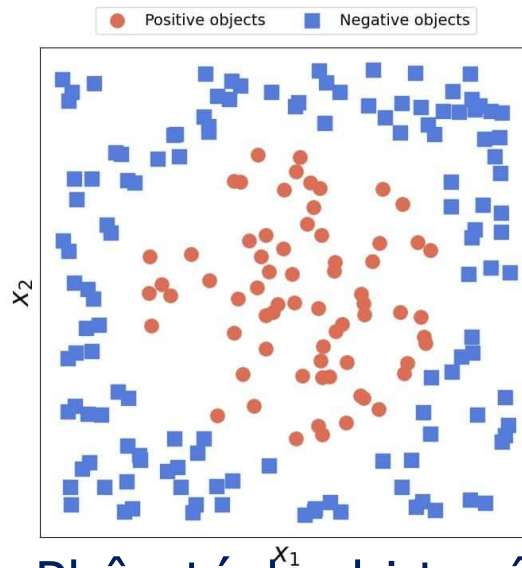
■ Vai trò:

- Nền tảng của neural network
- Cơ sở của deep learning



Phân tách tuyến tính

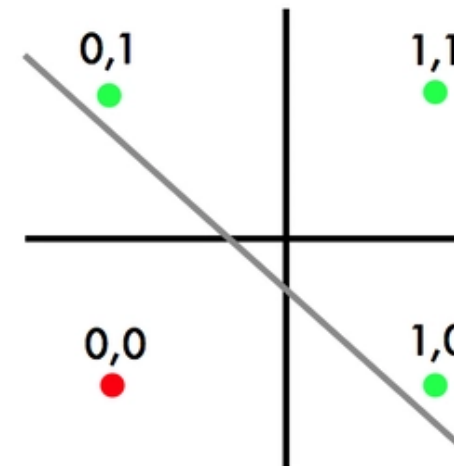
Deep Learning - Học Sâu



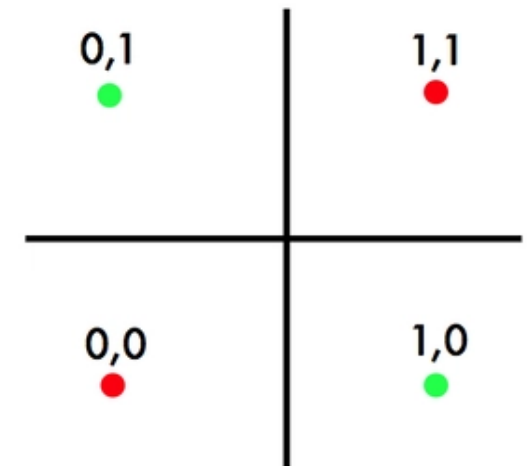
Phân tách phi tuyến

■ Hạn chế:

- Chỉ giải được bài toán **phân loại nhị phân tuyến tính** (AND, OR)



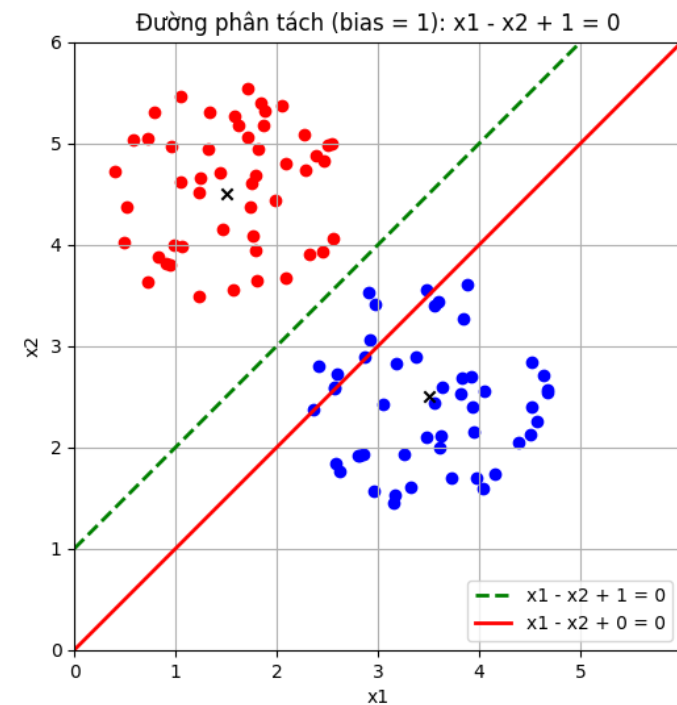
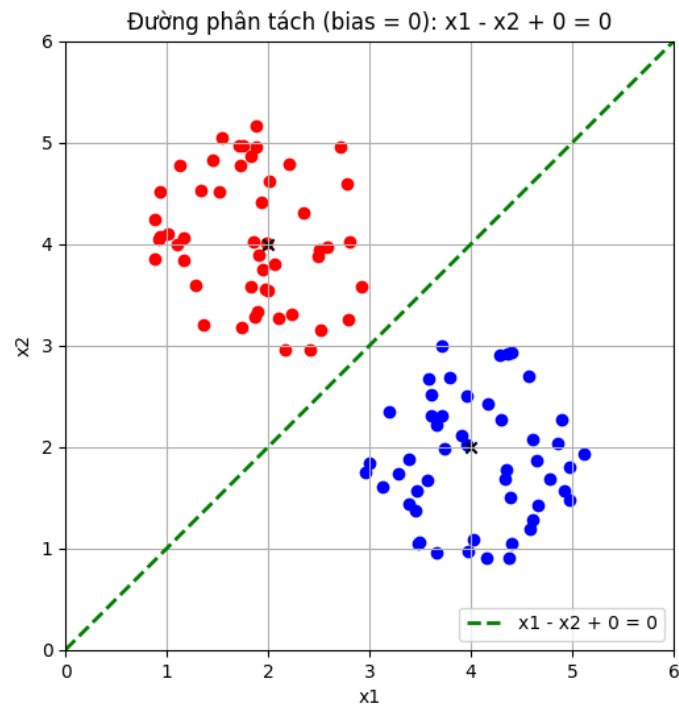
OR



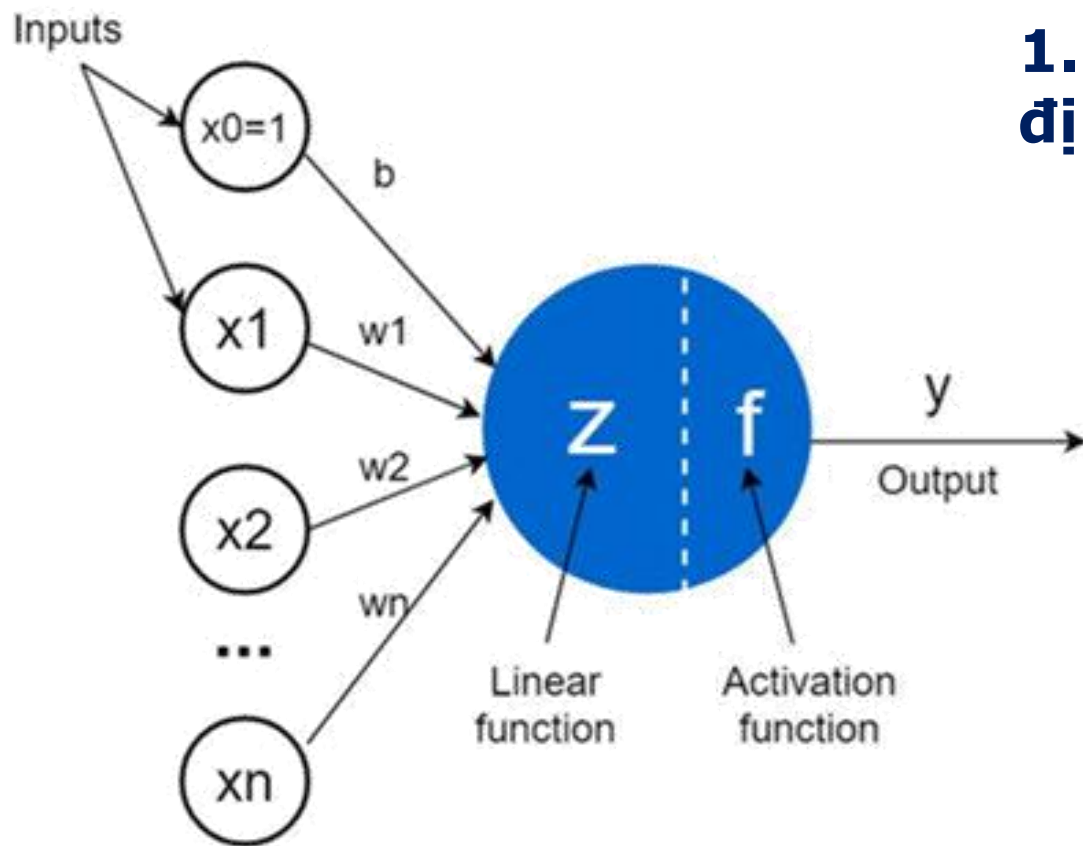
XOR

Vai trò của bias

- Dịch chuyển ranh giới quyết định
- Điều chỉnh ngưỡng kích hoạt
- Tăng tính linh hoạt của mô hình
- Giúp học tốt hơn trên dữ liệu thực tế



Vai trò của hàm kích hoạt: $y = f(w \cdot x + b)$



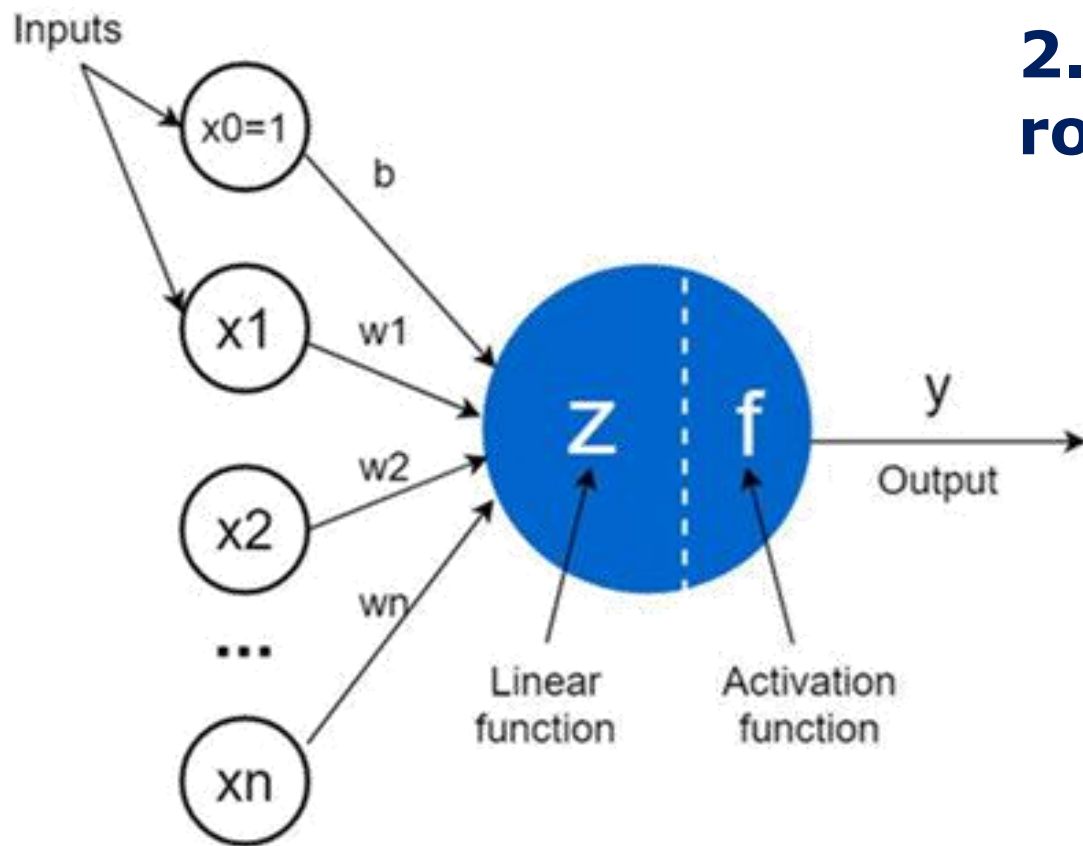
1. Biến tổng tuyến tính thành quyết định phân lớp

- Perceptron tính giá trị $z = w \cdot x + b$.
- Nếu không có hàm kích hoạt, z chỉ là một số thực.
- Hàm kích hoạt (ví dụ hàm bước (step func.)) sẽ chuyển z thành nhãn rời rạc:

$$f(x) = \begin{cases} 0, & x < 0 \\ 1, & x \geq 0 \end{cases}$$

→ Nhờ đó perceptron trở thành bộ phân loại nhị phân.

Vai trò của hàm kích hoạt: $y = f(w \cdot x + b)$



2. Thực hiện cơ chế “bật/tắt” nơ-ron

- Step func:

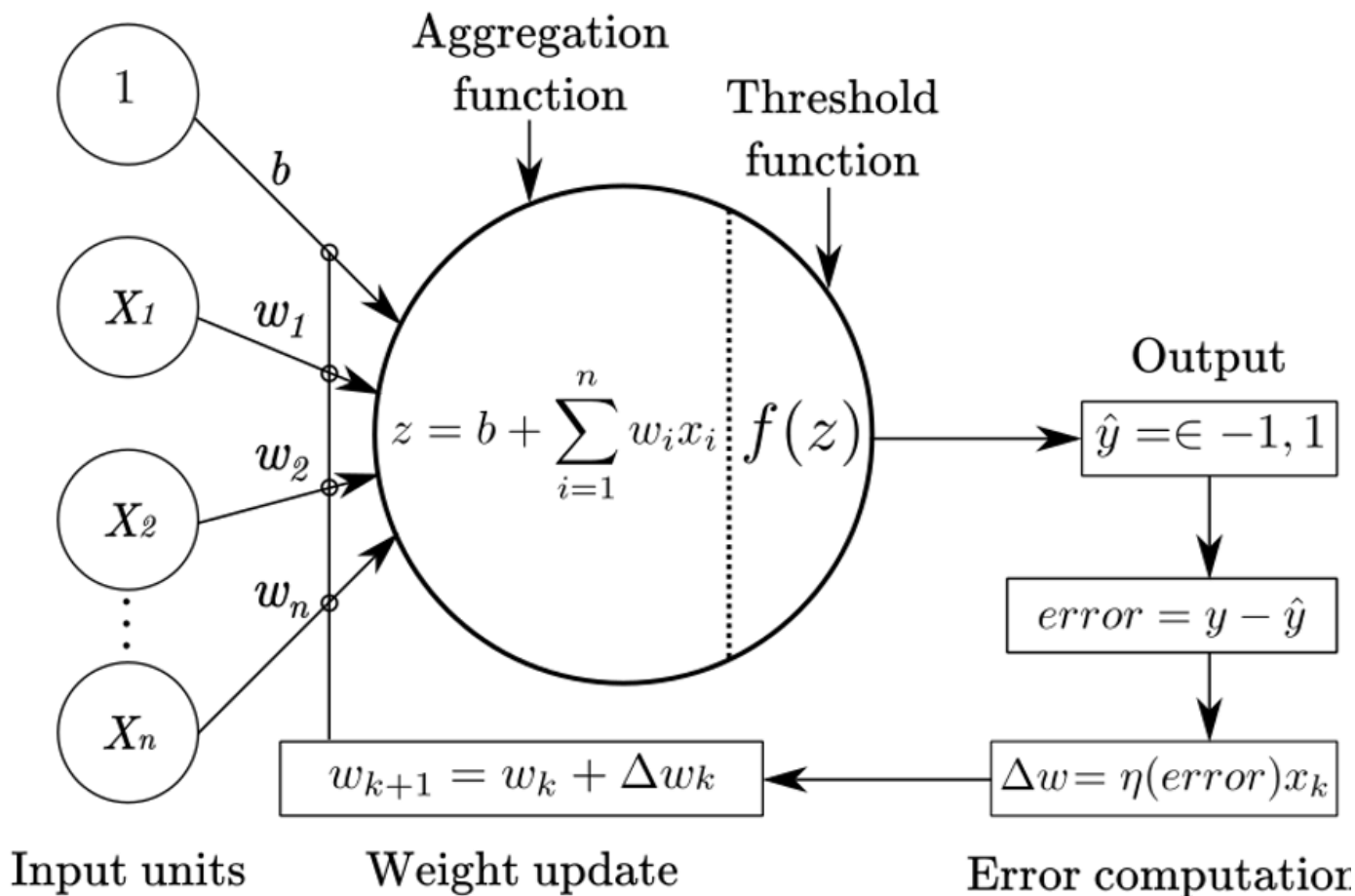
$$f(x) = \begin{cases} 0, & x < 0 \\ 1, & x \geq 0 \end{cases}$$

hoạt động như công tắc:

- Đủ điều kiện → kích hoạt (1)
- Không đủ → không kích hoạt (0)

→ Đây là ý tưởng ban đầu mô phỏng nơ-ron sinh học.

Huấn luyện Perceptron



$w = \text{random}()$

for epoch=1 to epochs:

for j=1 in len(training_set):

$$z = b + \sum_{i=1}^n w_i x_i$$

$$\hat{y} = f(z) \in \{-1, 1\}$$

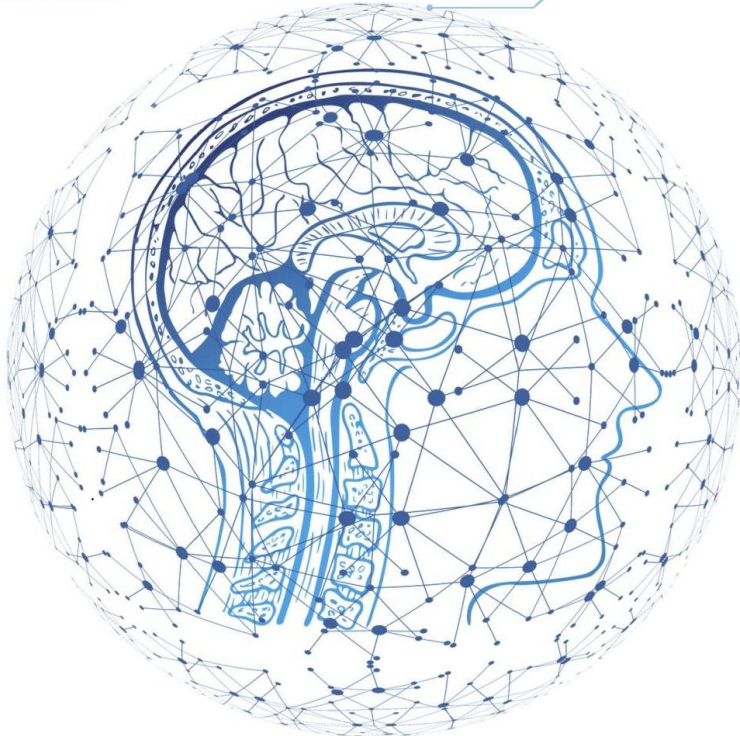
$$error = y - \hat{y}$$

$$\Delta w = \eta(error)x_k$$

$$w_{k+1} = w_k + \Delta w_k$$

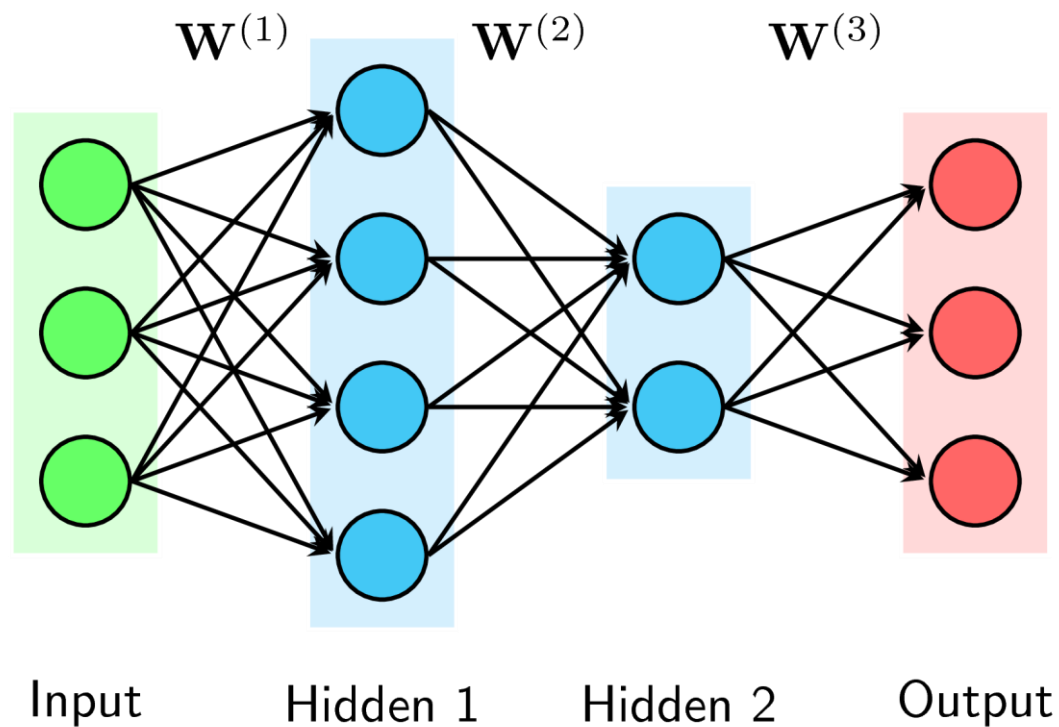
DEEP LEARNING

NỀN TẢNG LÝ THUYẾT CÁC MÔ HÌNH HỌC SÂU



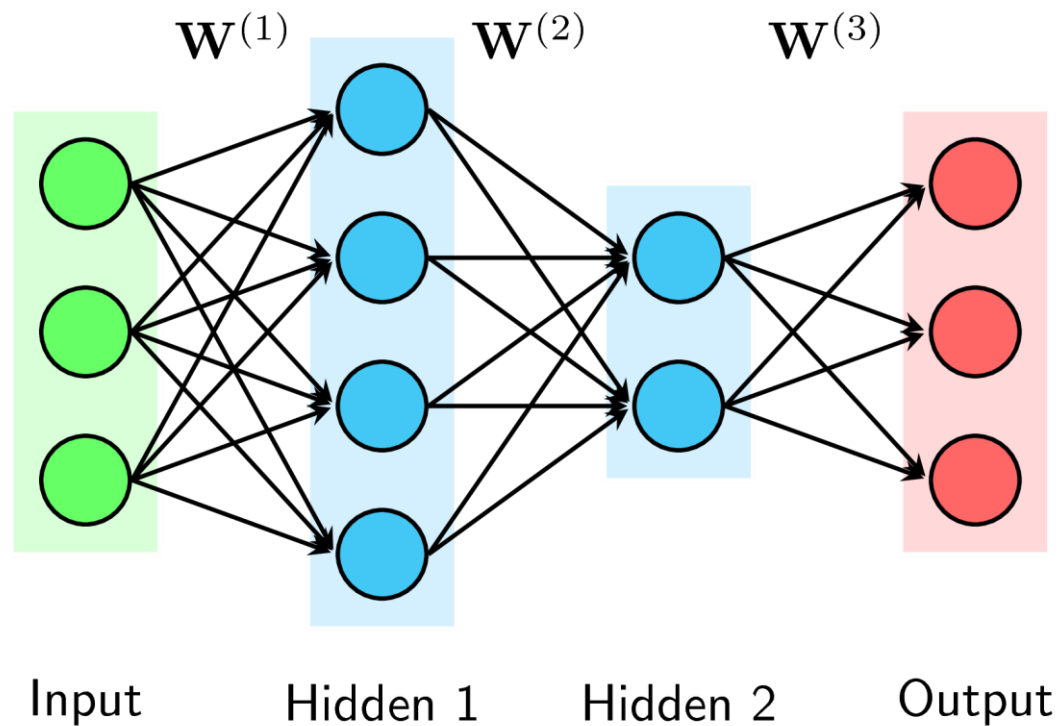
Mạng Neural Sâu (Deep Neural Networks)

Neural Network là gì?



- Neural Network là mô hình lấy cảm hứng từ não người
- Gồm các neuron (nút) và các kết nối có trọng số
- Nhận input, xử lý và tạo output

Từ Perceptron đến Deep Neural Network (DNN)



- Perceptron: 1 neuron
- Neural Network: là mạng neuron nói chung
- DNN: là mạng neuron có nhiều lớp ẩn
- Deep = nhiều lớp, nhiều tầng xử lý

Công dụng của DNN

- **Học các quan hệ phức tạp:** DNN có khả năng học các mối quan hệ phi tuyến rất phức tạp giữa input và output mà các mô hình đơn giản không làm được.
- **Học biểu diễn dữ liệu (representation learning):** DNN tự động học đặc trưng từ dữ liệu thay vì cần con người thiết kế thủ công.
 - Ví dụ:
 - Ảnh: pixel → cạnh → hình dạng → đối tượng
 - Văn bản: từ → ngữ cảnh → ý nghĩa

Công dụng của DNN

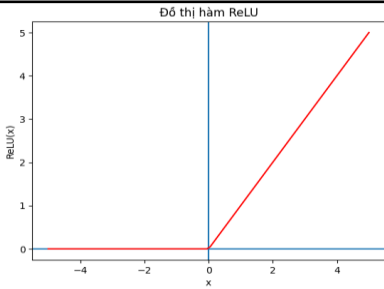
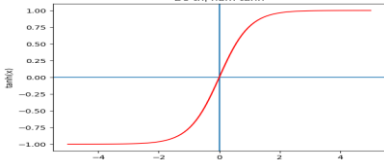
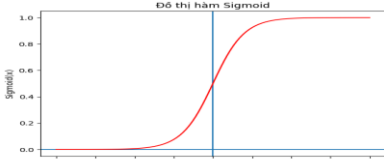
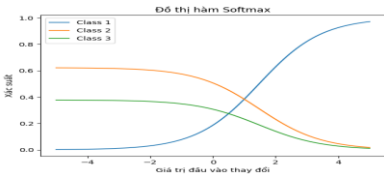
- **Xử lý dữ liệu lớn hiệu quả:** DNN hoạt động rất tốt khi có nhiều dữ liệu (big data), càng nhiều dữ liệu càng học tốt.
- **Ứng dụng đa dạng:** DNN được dùng trong rất nhiều lĩnh vực:
 - Nhận diện ảnh (Computer Vision)
 - Xử lý ngôn ngữ tự nhiên (NLP)
 - Nhận dạng giọng nói
 - Hệ gợi ý (recommendation systems)

Công dụng của DNN

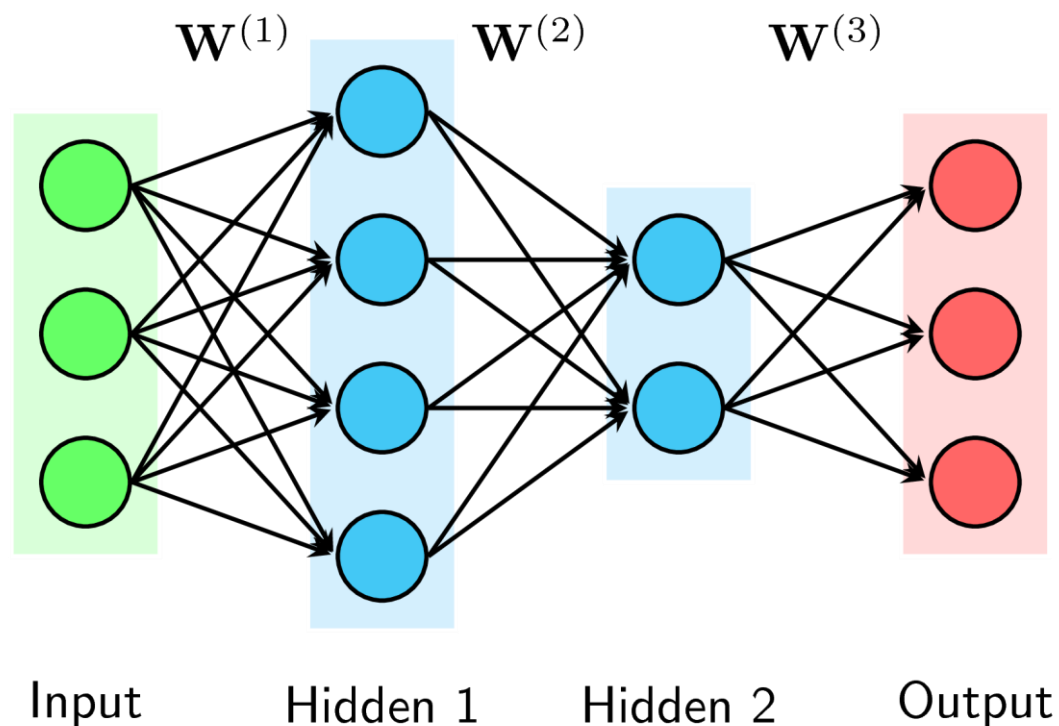
- **Tự động hóa các bài toán phức tạp:** DNN có thể thay thế các quy tắc thủ công bằng cách học trực tiếp từ dữ liệu.
- **Nền tảng của AI hiện đại:** Các hệ thống AI hiện nay (chatbot, xe tự lái, nhận diện khuôn mặt...) đều dựa trên DNN.

DNN giúp máy tính học đặc trưng phức tạp từ dữ liệu và giải quyết các bài toán mà trước đây rất khó hoặc không thể.

Các hàm kích hoạt thông dụng

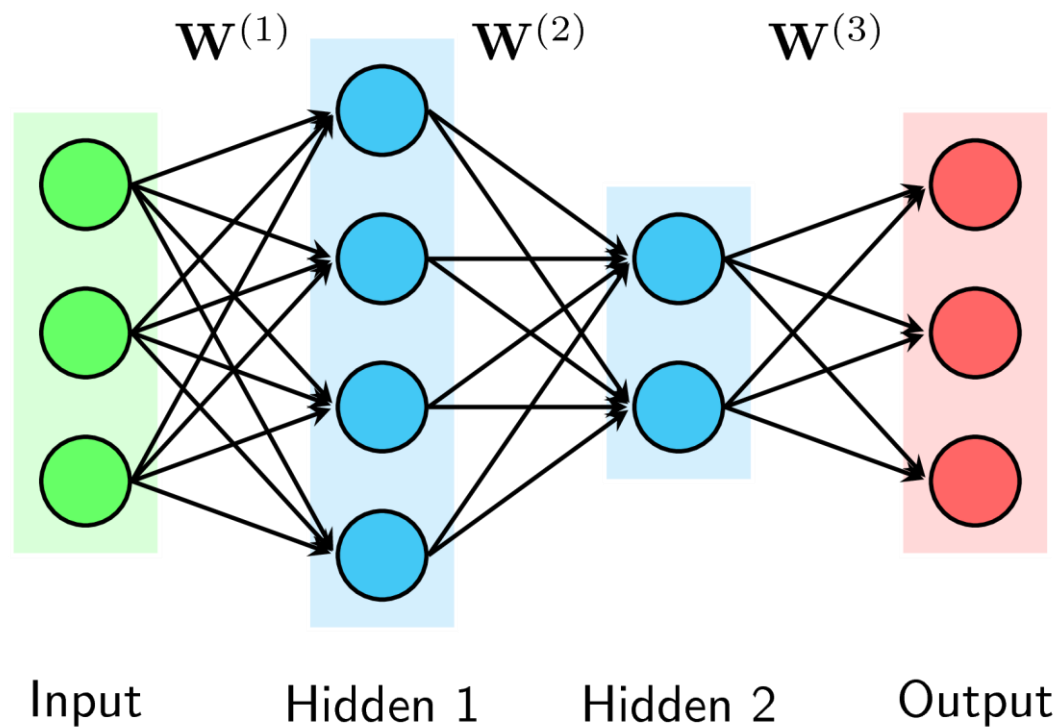
Hàm	Đồ thị	Công dụng	Layer Sử dụng	Bài toán Áp dụng
$\text{ReLU}(x) = \max(0, x)$	 Đồ thị hàm ReLU	Tạo phi tuyến, tính toán nhanh, giảm hiện tượng vanishing gradient (Gradient rất nhỏ)	Hidden layer	Hầu hết bài toán DL
$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	 Đồ thị hàm tanh	Chuẩn hóa về (-1,1), trung tâm tại 0	Hidden layer	RNN, bài toán cần dữ liệu có giá trị âm/dương
sigmoid $\sigma(x) = \frac{1}{1 + e^{-x}}$	 Đồ thị hàm Sigmoid	Biến giá trị về (0,1), biểu diễn xác suất	Output layer	Phân loại nhị phân
$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$	 Đồ thị hàm Softmax	Chuyển vector thành phân phối xác suất	Output layer	Phân loại đa lớp

Huấn luyện DNN



- Mục tiêu: Tìm bộ tham số W, b sao cho mô hình dự đoán đúng nhất, tức là làm nhỏ hàm mất mát: $Loss(y, \hat{y}) \rightarrow \min$
- Huấn luyện DNN:
 - Dùng thuật toán Backpropagation (lan truyền ngược)
 - Kết hợp với một thuật toán tối ưu (optimizer), phổ biến nhất là: Gradient Descent

Thuật toán Backpropagation + Gradient Descent



■ Ý tưởng chính:

- Forward: đưa dữ liệu đi từ input $\rightarrow$ output để tính dự đoán $\hat{y}$
- Tính $Loss(y, \hat{y})$: sai số giữa dự đoán và thực tế
- Backward (Backpropagation): lan truyền sai số ngược lại để tính gradient
- Cập nhật trọng số bằng Gradient Descent

Thuật toán Backpropagation + Gradient Descent

- Công thức cập nhật trọng số:

$$W := W - \eta \cdot \frac{\partial Loss}{\partial W}$$

- Trong đó:

- η (eta) là learning rate
- $\frac{\partial Loss}{\partial W}$ là gradient

- Các biến thể phổ biến của Gradient Descent:

- SGD (Stochastic Gradient Descent)
- Momentum
- RMSProp
- Adam (phổ biến nhất hiện nay)

Huấn luyện DNN:

Backpropagation tính gradient và Gradient Descent cập nhật trọng số

Thuật toán huấn luyện DNN chi tiết

- Bước 1. Khởi tạo tham số
 - Khởi tạo ngẫu nhiên trọng số W, b
- Bước 2. Lan truyền tiến (Forward Propagation) tính dự đoán $\hat{y}$
 - Với mỗi lớp l :
 - $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$
 - $a^{(l)} = f(z^{(l)})$
 - Trong đó:
 - $a^{(0)} = x$ (input)
 - f : hàm kích hoạt

Thuật toán huấn luyện DNN chi tiết

- Bước 3. Tính hàm mất mát: đo độ sai giữa dự đoán $\hat{y}$ và nhãn thật y
 - Ví dụ:
 - MSE: $Loss = (y - \hat{y})^2$
 - Cross-entropy
- Bước 4. Lan truyền ngược (Backpropagation):
 - Tính gradient của loss theo từng tham số
 - Dùng quy tắc dây chuyền (chain rule):

$$\frac{\partial Loss}{\partial W}, \frac{\partial Loss}{\partial b}$$

Thuật toán huấn luyện DNN chi tiết

■ Bước 5. Cập nhật tham số (Gradient Descent)

$$W = W - \eta \frac{\partial Loss}{\partial W} \qquad b = b - \eta \frac{\partial Loss}{\partial b}$$

- Trong đó:
 - η là learning rate

■ Bước 6. Lặp lại (Epoch)

- Lặp lại từ Bước 2 nhiều lần
- Cho đến khi loss hội tụ hoặc đạt số epoch mong muốn

Mã giả thuật toán huấn luyện DNN

- Khởi tạo ngẫu nhiên trọng số W, b
- for epoch = 1 $\rightarrow$ N:
 - for mỗi batch:
 - forward()
 - compute_loss()
 - backward()
 - update_parameters()

Epoch & Batch

- Epoch là 1 lần mô hình học qua toàn bộ tập dữ liệu.

- Ví dụ:

- Có 1000 mẫu dữ liệu
- Học hết 1000 mẫu → 1 epoch

→ Epoch = “Một lần học hết dữ liệu”

- Batch là một phần nhỏ của tập dữ liệu được đưa vào mô hình trong 1 lần cập nhật.

- Ví dụ:

- Có 1000 mẫu dữ liệu
- Batch size = 100

→ Mỗi lần học 100 mẫu
→ Cần 10 batch để hết dữ liệu

Epoch & Batch

- Epoch là 1 lần mô hình học qua toàn bộ tập dữ liệu.

- Ví dụ:

- Có 1000 mẫu dữ liệu
- Học hết 1000 mẫu → 1 epoch

→ Epoch = “Một lần học hết dữ liệu”

- Batch là một phần nhỏ của tập dữ liệu được đưa vào mô hình trong 1 lần cập nhật.

- Ví dụ:

- Có 1000 mẫu dữ liệu
- Batch size = 100

→ Mỗi lần học 100 mẫu
→ Cần 10 batch để hết dữ liệu

Epoch & Batch

- Số batch trong 1 epoch = số mẫu / batch size
- Ví dụ: 1000 mẫu, batch size = 100 $\rightarrow$ 1 epoch = 10 batch
- Trong thực tế thường dùng mini-batch: batch size nhỏ (32, 64, 128...) $\rightarrow$ Cân bằng giữa tốc độ huấn luyện và độ ổn định

So sánh các cách huấn luyện/học

Tiêu chí	Batch Gradient Descent (GD)	Mini-batch Gradient Descent (Mini-batch GD)	Stochastic Gradient Descent (SGD)
Số mẫu mỗi lần update	Toàn bộ dataset	Một nhóm nhỏ (32, 64, 128...)	1 mẫu
Số lần update / epoch	1	N/batch_size	N
Chi phí mỗi lần update	Rất cao	Trung bình	Thấp
Tốc độ mỗi lần update	Chậm	Nhanh	Rất nhanh
Tốc độ hội tụ	Chậm nhưng mượt	Nhanh và ổn định	Nhanh nhưng dao động
Độ ổn định gradient	Rất ổn định	Tương đối ổn định	Nhiều cao
Bộ nhớ yêu cầu	Cao	Trung bình	Thấp
Khả năng thoát local minimum	Kém	Tốt	Tốt
Khả năng tận dụng GPU	Kém	Tốt nhất	Kém
Phù hợp	Dataset nhỏ	Hầu hết bài toán DL	Online learning

Ví dụ Mini-batch Gradient Descent

1. Dataset ban đầu: Giả sử có 9 mẫu dữ liệu:

- [D1, D2, D3, D4, D5, D6, D7, D8, D9]

2. Chia thành các mini-batch: Giả sử batch size = 3 → chia thành:

- Batch 1: [D1, D2, D3],
- Batch 2: [D4, D5, D6],
- Batch 3: [D7, D8, D9]
- → 1 epoch = 3 mini-batch

Ví dụ Mini-batch Gradient Descent

3. Xử lý chi tiết từng mini-batch: Xét Batch 1: [D1, D2, D3]

- **Bước 1. Forward:** Đưa cả 3 mẫu vào mạng → Nhận 3 dự đoán: $\hat{y}_1, \hat{y}_2, \hat{y}_3$
- **Bước 2. Tính Loss:** Tính loss cho từng mẫu, sau đó lấy trung bình:

$$Loss = \frac{Loss_1 + Loss_2 + Loss_3}{3} \rightarrow \text{Đây là loss của mini-batch}$$

- **Bước 3. Backward:** Tính gradient của loss theo từng trọng số. Gradient này là tổng hợp từ cả 3 mẫu → Ổn định hơn so với dùng 1 mẫu
- **Bước 4. Update:** Cập nhật trọng số 1 lần duy nhất cho cả batch:

$$W := W - \eta \cdot \frac{\partial Loss}{\partial W}$$

Ví dụ Mini-batch Gradient Descent

4. Lặp lại cho các batch khác:

- Batch 2 $\rightarrow$ Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Update
- Batch 3 $\rightarrow$ Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Update
 $\rightarrow$ Kết thúc 1 epoch

■ So sánh nhanh:

- SGD: Batch size = 1 $\rightarrow$ 9 lần update/epoch
- BGD: Batch size = 9 $\rightarrow$ 1 lần update /epoch
- Mini-batch: Batch size = 3 $\rightarrow$ 3 lần update/epoch

